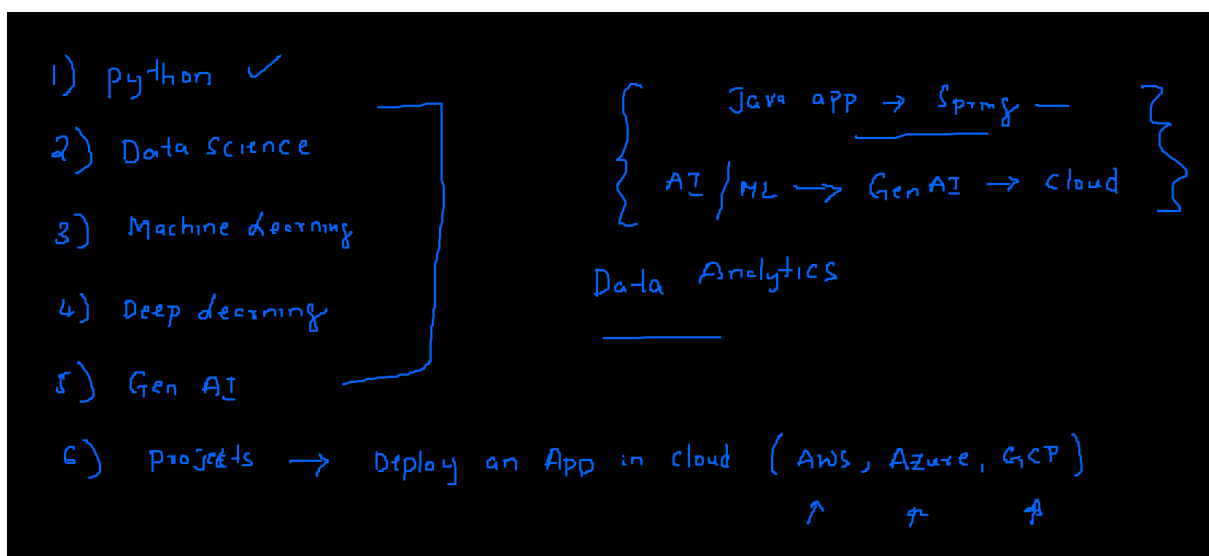


Contents

Course Content	1
Links.....	1
17 th Nov 2025	2
1 st Dec 2025.....	2
Flow Control.....	2
1. Conditional Statements (Decision Making)	2
2. Looping Statements (Iteration)	4
3. Control Transfer Statements (Jump Statements)	5

<https://meet.google.com/qku-vvga-eez>

Course Content



Links

<https://www.python.org/downloads/>

python --version

python3 --version

<https://www.anaconda.com/download>

<https://jupyter.org/>

17th Nov 2025

?

Take the input as (1) Item (2) Qty (3) calculate the total and display	Item	Qty	price	TOTAL
	pencil	2	10	20
	Sketch	3	20	60
	pencil	5	5	25
			Amount-payable	105

1st Dec 2025

Flow Control

Flow control, or control flow, is the order in which statements are executed in a program. In Python, it's managed using **control structures** that allow the program to make decisions, repeat actions, and jump between code blocks.

There are three main categories of flow control statements:

1. Conditional Statements (Decision Making)

These statements execute code only if a certain condition is met.

Statement	Purpose	Example
if	Executes a block of code if the condition is True .	if x > 10:
elif	Short for "else if." Checks a new condition if the previous if or elif conditions were False .	elif x == 10:
else	Executes a block of code if all preceding if and elif conditions are False .	else:

x = 15

if x > 20:

 print("x is large")

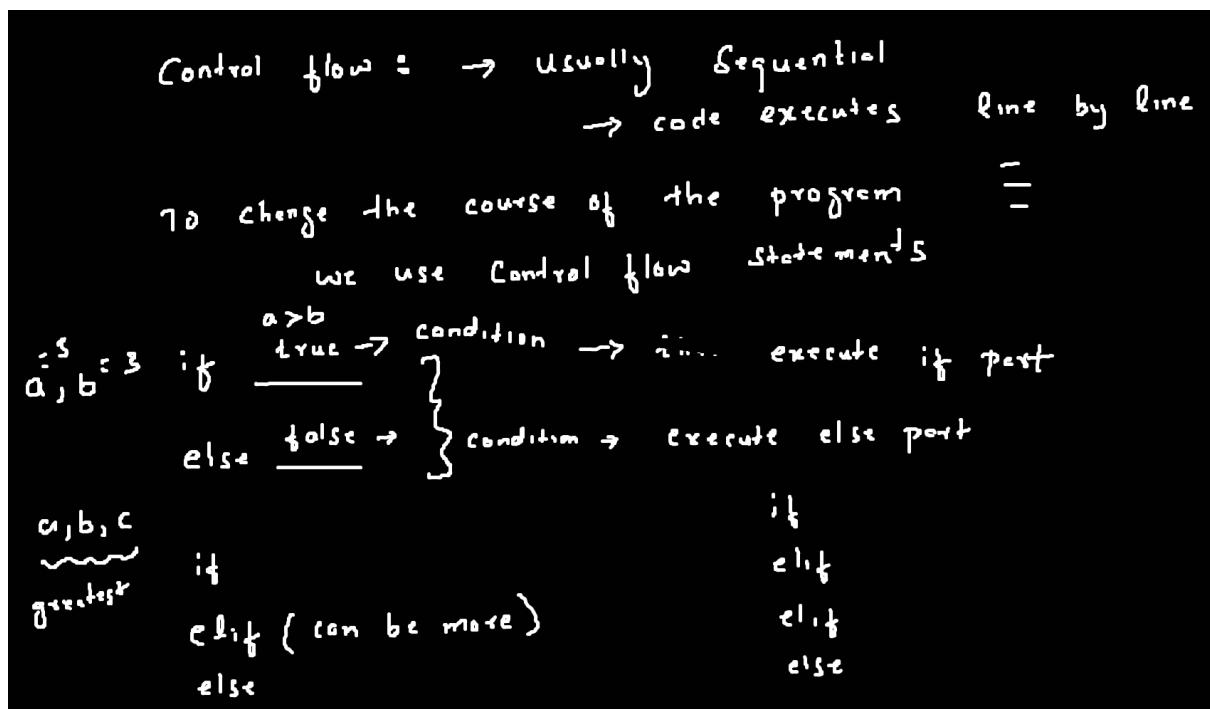
elif x > 10:

 # This condition is True, so this block executes

 print("x is medium")

else:

 print("x is small")



```
e Edit View Run Kernel Settings Help
+ ✂ 📄 ▶ ■ ↺ ▶▶ Code ▾ JupyterLab 🌐 🐍 Pyt
```

```
[12]: #if statement
num1 = 200
if num1 < 100:
    print("num1 is less than 100")
    print("The value of num1 is ", num1)
```

(200 < 100) → False
condition returns a boolean value
↳ True | False

```
[14]: #if statement
num1 = 20
if num1 < 100:
    print("num1 is less than 100")
    print("The value of num1 is ", num1)
```

True | False
20 < 100 True

```
num1 is less than 100
The value of num1 is 20
```

2. Looping Statements (Iteration)

These statements execute a block of code multiple times, either for a fixed number of times or until a condition is met.

A. for loop

The for loop is used for **iterating** over a sequence (like a list, tuple, dictionary, set, or string) or other iterable objects. It is typically used when you know the number of iterations.

Iterating over a list

```
fruits = ["apple", "banana", "cherry"]
```

```
for fruit in fruits:
```

```
    print(f"I like {fruit}")
```

Iterating a fixed number of times using range()

```
for i in range(3): # range(3) produces 0, 1, 2
```

```
    print(i)
```

B. while loop

The while loop repeatedly executes a block of code as long as the specified **condition is True**. It is typically used when the number of iterations is unknown beforehand.

```
count = 0
```

```
while count < 3:
```

```
    print(f"Count is {count}")
```

```
    count += 1 # Increments count, eventually making the condition False
```

3. Control Transfer Statements (Jump Statements)

These statements alter the normal execution flow *within* a loop.

Statement	Purpose
break	Immediately terminates the entire loop (both for and while), skipping the rest of the code in the loop and moving execution to the statement immediately following the loop.
continue	Skips the rest of the code inside the current iteration of the loop, moving control back to the loop's beginning to start the next iteration.
pass	A null operation . It does nothing but acts as a placeholder where a statement is syntactically required but you don't want any code to execute.

```
for num in range(10):
```

```
    if num == 3:
```

```
        # Skips the rest of the code for num=3, moves to num=4
```

```
        continue
```

```
    if num == 7:
```

```
        # Immediately exits the entire loop
```

```
        break
```

```
    print(num)
```

```
# Output: 0, 1, 2, 4, 5, 6
```

GitHub – 16th May 2026

